

1-Month System Design Masterclass Roadmap

This structured learning curriculum is carefully compiled using core engineering methodologies from the open-source **GitHub System Design Primer** (by Donne Martin). Designed for students and aspiring engineers, it consolidates complex large-scale architecture principles into a high-intensity, 30-day timeline. The final two weeks are explicitly reserved for hands-on operational practice across both baseline and advanced real-world system patterns.

Weekly Breakdown Overview

Week	Focus Theme	Primary Goal
Week 1	Foundations & Scalability Primitives	Master core abstractions, performance indicators, and basic infrastructure elements.
Week 2	Data Tiers & Distributed Systems Theory	Deconstruct data storage models, caching, data consistency trade-offs, and messaging.
Week 3	Pure Practice: Baseline Fundamental Systems	Apply the 4-step interview framework to standard, predictable designs.
Week 4	Pure Practice: High-Complexity Platforms	Deconstruct global, highly distributed services with intense concurrency requirements.

Week 1: Foundations & Scalability Primitives

Focus on understanding how computers communicate across networks, how workloads are distributed across infrastructure, and how to analyze system boundaries objectively.

Day-by-Day Syllabus

- **Day 1: Design Evaluation Metrics & Scale Frameworks**
 - Performance vs. Scalability: Understanding bottlenecks under increasing load.
 - Latency vs. Throughput: Optimizing transaction speeds versus absolute delivery

- capacity.
 - Back-of-the-envelope estimations: Calculating memory, storage, and CPU bandwidth requirements up front.
- **Day 2: Application Tiers & Scaling Methods**
 - Vertical Scaling (Scale Up) vs. Horizontal Scaling (Scale Out).
 - Stateless Application Tier: Decoupling state to easily scale compute instances out.
- **Day 3: Network Protocols & Communication Patterns**
 - Understanding layers: IP, TCP, and UDP characteristics.
 - Application Layer Protocols: HTTP, HTTPS, WebSockets for bi-directional communication, and RPC/gRPC.
- **Day 4: Load Balancing Infrastructure**
 - Hardware vs. Software load balancers (Nginx, HAProxy, AWS ALB).
 - Routing algorithms: Round Robin, Least Connections, IP Hash, and Consistent Hashing.
 - Layer 4 (Transport) vs. Layer 7 (Application) routing choices.
- **Day 5: Edge Networks & Content Delivery Systems**
 - Content Delivery Networks (CDNs): Push vs. Pull strategy for serving static and dynamic assets.
 - Reverse Proxies vs. Forward Proxies: Security, termination of SSL, and structural differences.
- **Day 6: Domain Name Services & Edge Routing**
 - DNS Architecture: Name servers, record types (A, AAAA, CNAME), and Anycast routing.
- **Day 7: Weekly Revision & Flashcard Review**
 - Review system design definitions and complete the core conceptual flashcards outlined in the System Design Primer repo.

Week 2: Data Tiers & Distributed Systems Theory

Shift focus to where data lives, how consistency is managed across multiple data nodes, and how decoupled architectures process jobs concurrently.

Day-by-Day Syllabus

- **Day 8: Relational Databases & Scaling Operations**
 - Relational Database Management Systems (RDBMS): ACID transactions and structured optimization.
 - Master-Slave (Read Replication) and Master-Master multi-write replication models.
- **Day 9: Non-Relational Storage Models & Database Partitioning**
 - NoSQL Categories: Key-Value, Document, Wide-Column, and Graph stores. When to opt out of relational properties.
 - Database Sharding: Horizontal partitioning strategies, shard key selection, and re-sharding challenges.
- **Day 10: Theoretical Guardrails: CAP & PACELC Theorems**
 - CAP Theorem: Deciding between Availability and Partition Tolerance (AP) vs. Consistency and Partition Tolerance (CP).

- Consistency Patterns: Weak consistency, Eventual consistency, and Strong consistency trade-offs.
- **Day 11: Enterprise Caching Layers**
 - Application vs. Database caching (Redis, Memcached architectures).
 - Cache Invalidation Strategies: Write-through, Write-behind (Write-back), and Refresh-ahead policies.
 - Cache eviction schemas: LRU, LFU, FIFO.
- **Day 12: Asynchronous Systems & Event Streaming**
 - Message Queues vs. Pub/Sub Engines (RabbitMQ, Apache Kafka).
 - Decoupling slow processing workflows from synchronous request-response cycles.
- **Day 13: High-Scale File & Storage Frameworks**
 - Object Storage abstractions (AWS S3) vs. Block Storage.
 - Distributed File Systems fundamentals (HDFS, Google File System concepts).
- **Day 14: Mid-Way Checkpoint & Architecture Exercises**
 - Incorporate basic components together. Practice combining a Load Balancer -> Web Server Pool -> Cache -> Database Cluster into structural topology diagrams.

The 4-Step System Design Approach (For Practice Weeks)

For every system design question in Weeks 3 and 4, strictly apply the system design primer's systematic four-step blueprint:

1. **Step 1: Outline Use Cases, Constraints, and Assumptions:** Define scope, core features, expected users, Read vs. Write scale ratios, and target data sizing profiles.
2. **Step 2: Create a High-Level Design:** Sketch an end-to-end component blueprint showing core clients, load managers, application entry nodes, and persistence tiers.
3. **Step 3: Design Core Components:** Deep-dive into database schema design, specific API signatures, hash generation strategies, or specific cache invalidation timelines.
4. **Step 4: Scale the Design:** Identify operational single points of failure, insert redundant layers, introduce load regulation components, and deploy optimal partitioning keys.

Week 3: Pure Practice: Fundamental Baseline Systems

Focus on predictable, deterministic business logic questions. Learn to execute clean data indexing, simple high-velocity reads, and precise write patterns cleanly.

Day-by-Day Case Studies

- **Day 15: Design a URL Shortening Service (e.g., Bitly)**
 - Core Topics: Base62 encoding, MD5 hashing, collision handling, and optimizing for massive read redirection using caches.
- **Day 16: Design a Scalable Pastebin Engine**

- Core Topics: Object storage integrations for text payloads, expiration metadata management, and database schema creation.
- **Day 17: Design an Enterprise API Rate Limiter**
 - Core Topics: Token Bucket, Leaky Bucket, and Sliding Window Log algorithms implemented on Redis clusters.
- **Day 18: Design a Web Crawler infrastructure**
 - Core Topics: Breadth-First Search (BFS) graphs, deduplication indices via Bloom Filters, robots.txt compliance, and queue workers.
- **Day 19: Design a Read-Heavy Key-Value Store**
 - Core Topics: In-memory hash maps, disk serialization fallback, and high-availability multi-node consensus basics.
- **Day 20: Design a Global Content Content Storage Layer (CDN Analytics)**
 - Core Topics: Continuous log aggregation pipelines, data windowing counters, and writing to analytical column stores.
- **Day 21: Week 3 Retrospective & Peer Evaluation**
 - Re-trace your architecture blueprints from the past six days. Identify where single database points of failure remained unaddressed.

Week 4: Pure Practice: Complex Distributed Platforms

Deconstruct massive multi-user concurrent systems with high cross-boundary data footprints, real-time messaging, and globally distributed state synchronization.

Day-by-Day Case Studies

- **Day 22: Design a Distributed Financial Ledger System (e.g., Mint / Stripe Core)**
 - Core Topics: Absolute data consistency patterns, idempotent request handling, transaction replay logs, and dual-entry bookkeeping rules.
- **Day 23: Design a Real-Time Instant Messaging Service (e.g., WhatsApp / Slack)**
 - Core Topics: Keeping millions of active WebSocket connections, ephemeral message queue patterns, user presence systems, and database clustering.
- **Day 24: Design a Social Media Feed Generator (e.g., Twitter/X Timeline)**
 - Core Topics: Fan-out on Write (Push model) vs. Fan-out on Read (Pull model), hybrid caching systems for celebrity accounts, and fast pagination.
- **Day 25: Design a Video Streaming Network (e.g., Netflix / YouTube)**
 - Core Topics: Video chunk ingestion pipelines, global CDN content distribution topologies, adaptive bitrate streaming, and asymmetric user read profiles.
- **Day 26: Design a Distributed Search Autocomplete System (Typeahead)**
 - Core Topics: Building and updating a distributed Trie data structure, log gathering microservices, and front-end edge caching patterns.
- **Day 27: Design a Proximity/Geo-Spatial Marketplace Server (e.g., Uber or Yelp)**
 - Core Topics: High-velocity geo-indexing frameworks (Quadrees, Geohashes, Google H3 indices), and handling fast real-time coordinate updates.
- **Day 28: Mock Session: Comprehensive Design Synthesis**
 - Simulate an unpredictable end-to-end production launch scenario under arbitrary scale (e.g., scaling up from 10k to 10M active users overnight).

- **Days 29 & 30: Final Polish & Architectural Refinement**
 - Synthesize systemic trade-offs: SQL vs NoSQL selections, Eventual Consistency vs Availability boundaries, and compile your playbook of standard architectural failure handling mechanisms.